Introduction aux Bases de Données Relationnelles

Cours 1 : Modèle Entité-Association

UFR 919 – Licence Informatique

LU2IN009

Informations pratiques

Responsable de l'UE :

- Camelia Constantin

Chargés de cours 2026 :

- Mercredi 8h45-10h30 : Bernd AMANN
- Vendredi 8h45-10h30 : Camelia CONSTANTIN

Informations supplémentaires : voir la page Moodle

Où trouve-t-on des Bases de Données ?

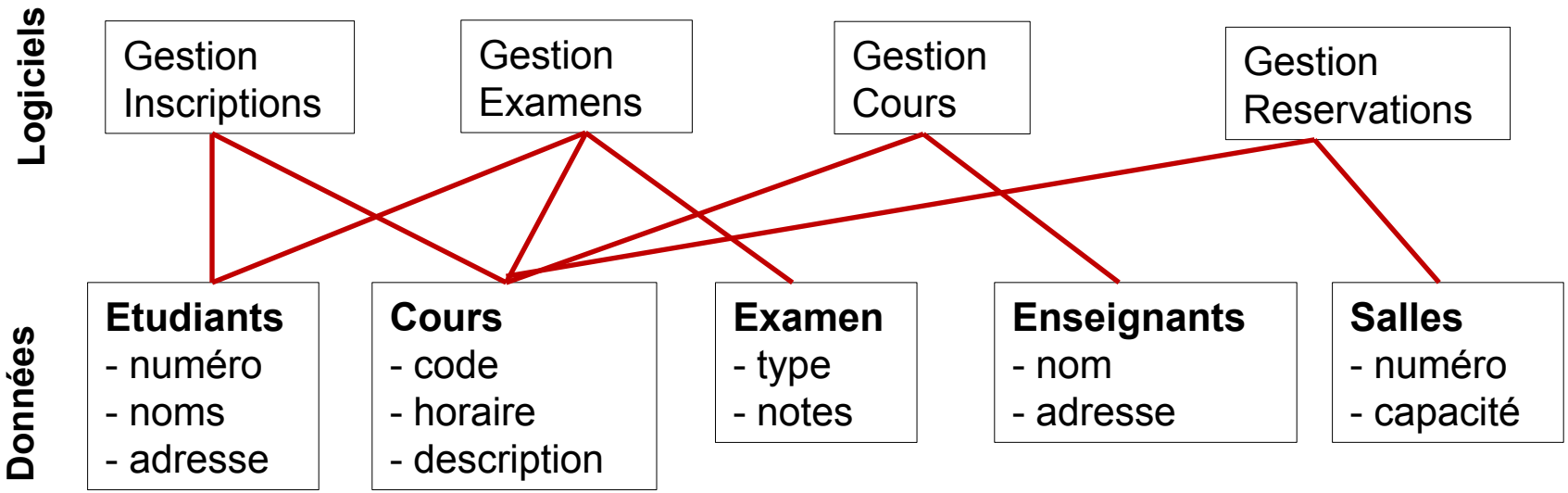
Secteur	Applications
Web & Médias	E-commerce, Streaming Audio/Video, Réseaux Sociaux
Finance	Banques, Bourse, Systèmes de Paiement
Service Publique	Impôts, Police, Education, Santé
Industrie & Commerce	Stock, Facturation, IOT (objets connectés), Télématique
Transport & Logistique	GPS & Navigation, Suivi de colis, Applications VTC
Tourisme & Voyage	Réservations d'hôtels/vols, Guides touristiques virtuels, Comparateurs de prix
Énergie & Environnement	Smart Grids, Gestion d'énergie, Prédiction Météo
Sciences et Education	Expérimentations scientifiques, Archives Documentaires

Mais qu'est-ce qu'une Base de Données ?

Un fichier, une clé USB, un programme ?

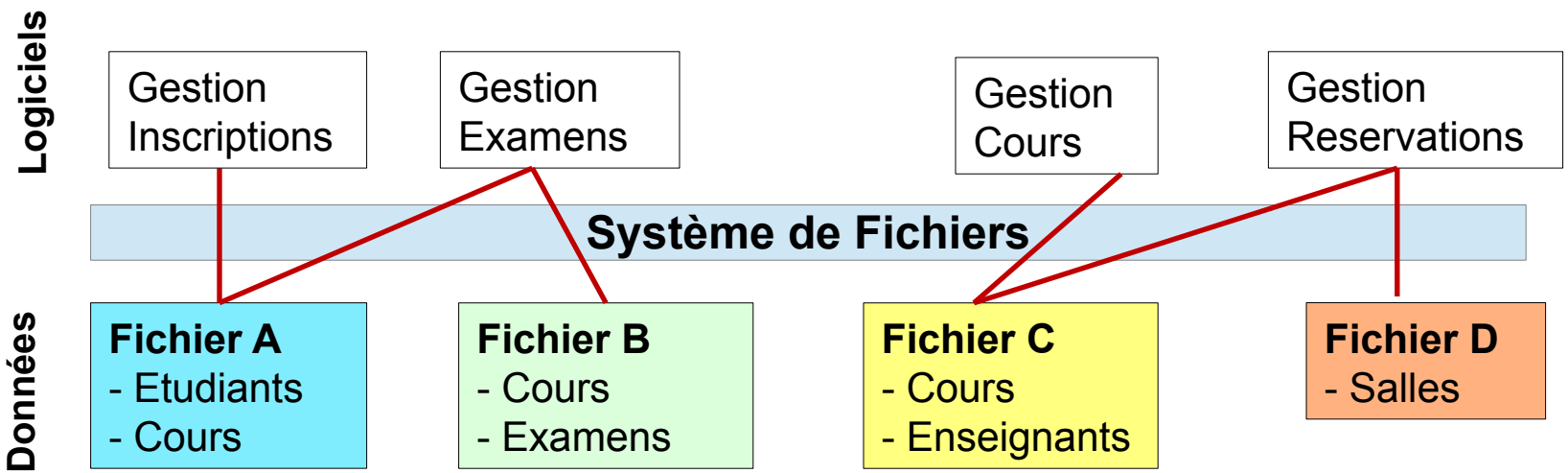
Gestion de Données

- **Accès** : chercher, insérer, modifier, effacer des données
- **Partage** : partager de données entre plusieurs applications/logiciels
- **Stockage** : éviter de stocker la même information plusieurs fois
- **Qualité** : éviter des erreurs dans les données (cohérence)
- **Performance** : être capables de gérer des grands volumes de données partagées par de nombreux utilisateurs
- **Sécurité** : protéger des données sensibles



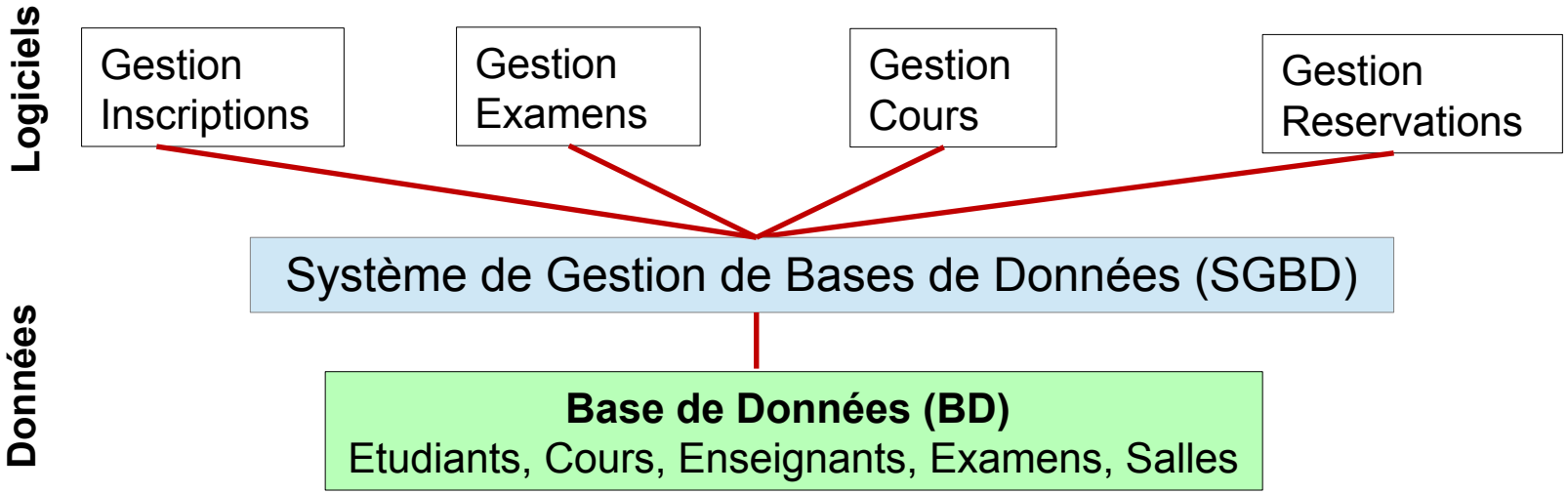
Systèmes de Fichiers

- **Accès** : lecture et écriture de fichiers entiers (accès séquentiel)
- **Partage** : accès simultané aux fichiers sans contrôle (conflits lectures/écritures)
- **Stockage** : la même information est souvent répétée dans plusieurs fichiers
- **Qualité** : un logiciel erroné peut facilement introduire des erreurs
- **Performance** : la gestion de grands fichiers est généralement inefficace
- **Sécurité** : protection limitée de données (droit accès aux fichiers entiers)



Système de Gestion de Bases de Données (SGBD)

- **Accès** : langages spécialisés d'interrogation et de gestion de données (SQL)
- **Partage** : contrôle centralisé des accès aux données
- **Stockage** : gestion de stockage indépendante des applications (logiciels)
- **Qualité** : gestion globale de cohérence de données (schémas, pannes, concurrence)
- **Performance** : optimisation d'accès aux données (indexation, cache, ...)
- **Sécurité** : protection fine des données sensibles



Architecture Clients / Serveur

Base de Données (BD) : ensemble structuré de données liées, représentant une réalité spécifique (ex: un inventaire, un réseau social).

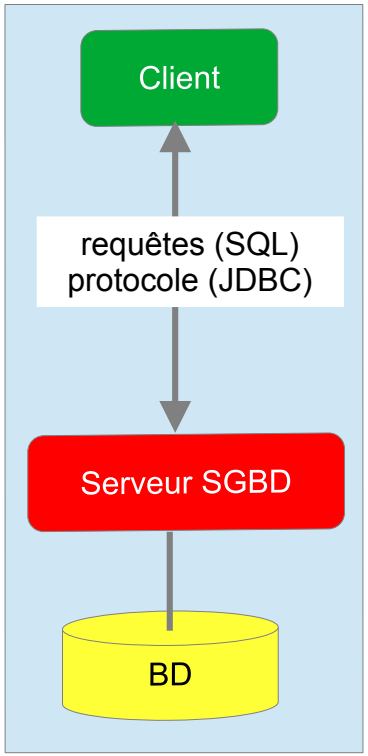
Architecture client/serveur :

- ***Serveur*** : instance d'un **SGBD** qui contrôle l'accès à la BD
- ***Clients*** : applications / **programmes** (Python, Java, C, …) qui communiquent avec le serveur
- ***Communication* :** échange de ***requêtes*** bases de données (SQL, protocole JDBC)

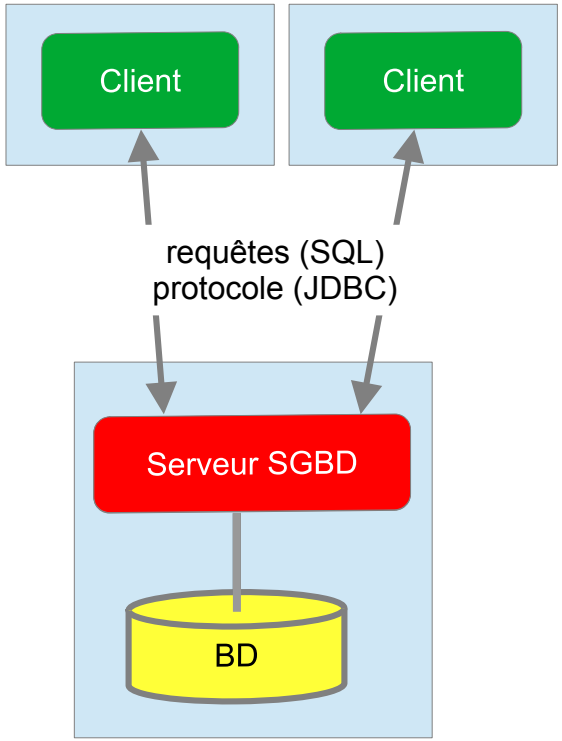
Le serveur SGBD :

- Reçoit les requêtes (interrogation, mises-à-jour) de tous les clients (applications)
- Ordonne, optimise et exécute les requêtes reçues
- Gère les conflits d'accès, les pannes, les requêtes erronées, …
- Renvoie les réponses aux clients

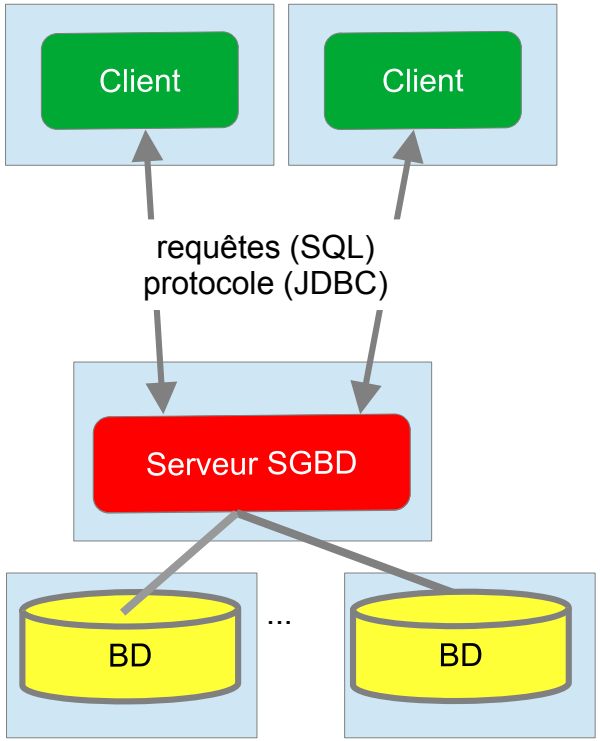
Configurations Clients / Serveur



Client/serveur sur
le même nœud
(serveur local)

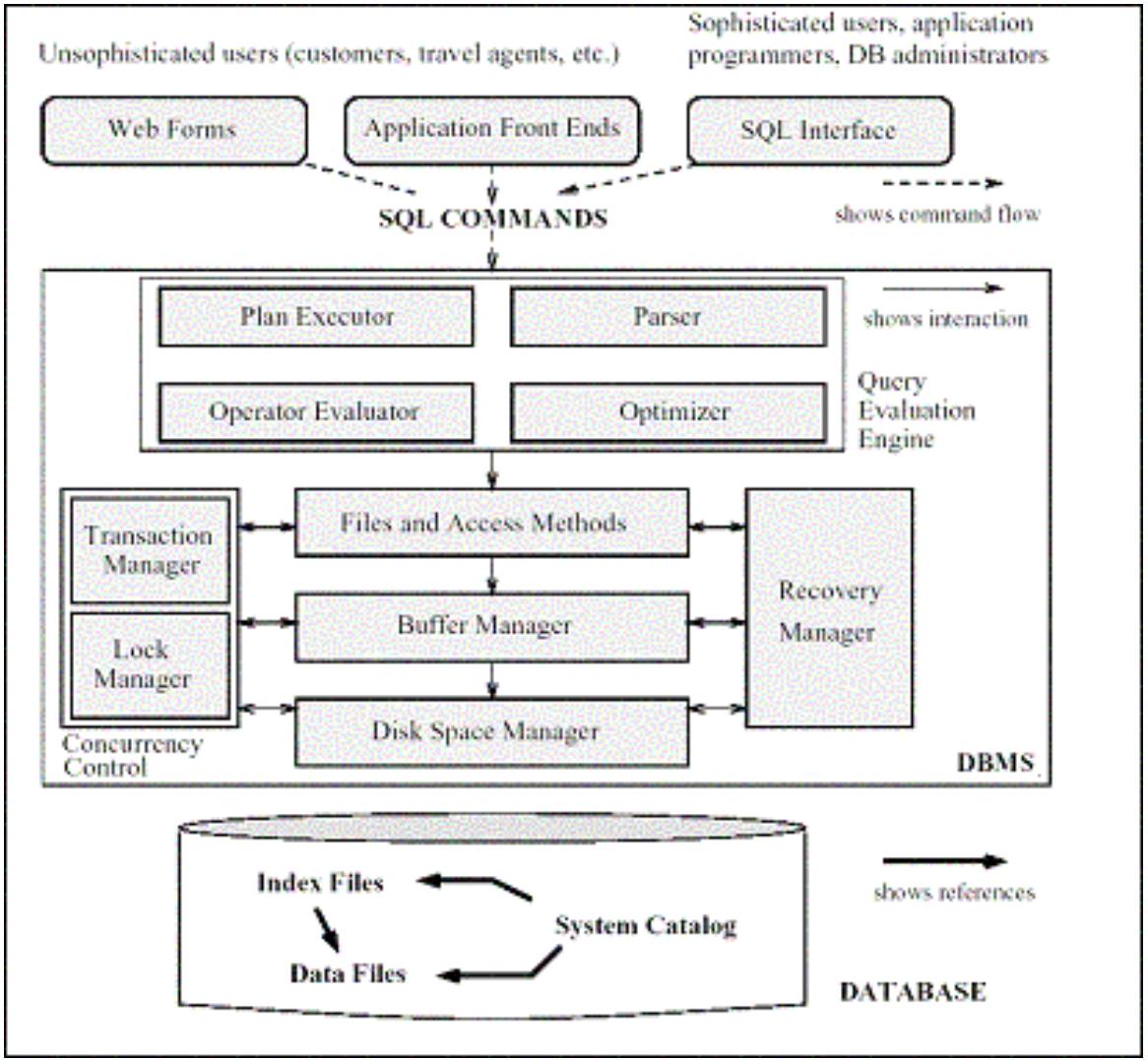


Client/serveur sur
nœuds distincts
(service BD)



BD distribuée
(cluster, scalabilité)

Architecture d'un SGBD



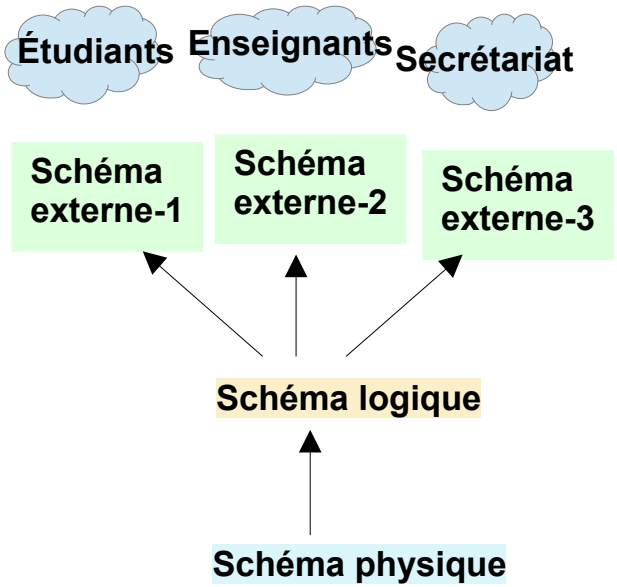
Cours L3 : LU3IN008

Source: Ramakrishnan & Gehrke

Utilisateurs d'un SGBD

- **L’architecte de l’information** : conçoit le modèle conceptuel des données et collabore étroitement avec les utilisateurs finaux afin de comprendre leurs besoins.
- **Le concepteur de bases de données :** transforme le modèle conceptuel des données en un modèle logique
- **L’administrateur de bases de données (DBA)** est responsable de la mise en place, de l’exploitation, de la sécurité et de la surveillance de la base de données.
- **Le développeur applicatif** développe des applications exploitant la base de données, à l’aide de langages de programmation tels que Java ou Python, afin de faciliter l’accès aux données pour les utilisateurs finaux.
- **L’utilisateur final** utilise ces applications pour effectuer des opérations spécifiques sur la base de données (consultation, mise à jour, analyse, etc.).
- **Le développeur de SGBD** conçoit et développe le logiciel du SGBD (par exemple chez IBM ou Oracle).

Architecture à 3 niveaux



Niveau externe (accessible à l'utilisateur / applications)

Quelles données est visible à chaque programme ?

→ **schéma externe (vues)**

Niveau logique (accessible au concepteur/programmeur)

Comment les données sont organisées ?

→ **schéma logique (tables)**

Niveau interne (accessible au concepteur et à l'administrateur)

Comment les données sont stockées ?

→ **schéma physique (index, fichiers, matériel)**

Indépendance
Niveau Logique - Interne

Principe :

- Dissociation du niveau logique (schéma logique) et du niveau interne (schéma physique).

Objectifs :

- Pouvoir modifier le schéma physique (fichiers, index, matériel) sans impacter le schéma logique (tables) ou les applications.
- *Améliorer les performances du système sans changer l'organisation logique des données.*

Mécanisme :

- Le SGBD implémente une interface qui traduit les structures et opérations logiques en structures de données et fonctions exécutables.

Indépendance
Niveau Externe - Logique

Principe : Dissociation du niveau externe (schémas externes) et du niveau logique (schéma logique).

Objectif :

- Pouvoir modifier le schéma logique (tables) sans altérer les schémas externes (vues) ni les programmes d'application existants.
- *Assurer la pérennité et la stabilité des logiciels applicatifs face aux évolutions structurelles de la base de données.*

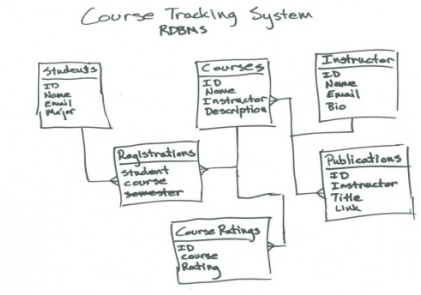
Mécanisme :

- Les ***vues*** agissent comme une interface protectrice. En cas d'évolution du modèle logique global, seule la *définition de la vue* est mise à jour pour maintenir une présentation constante des données aux applications.

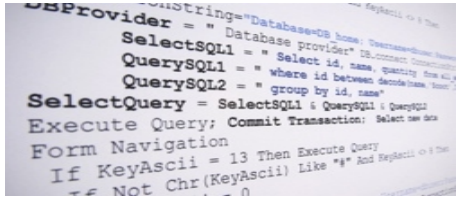
Comment construire une base de données

1) Analyse des besoins : *observer le monde réel identifier les informations pertinentes*

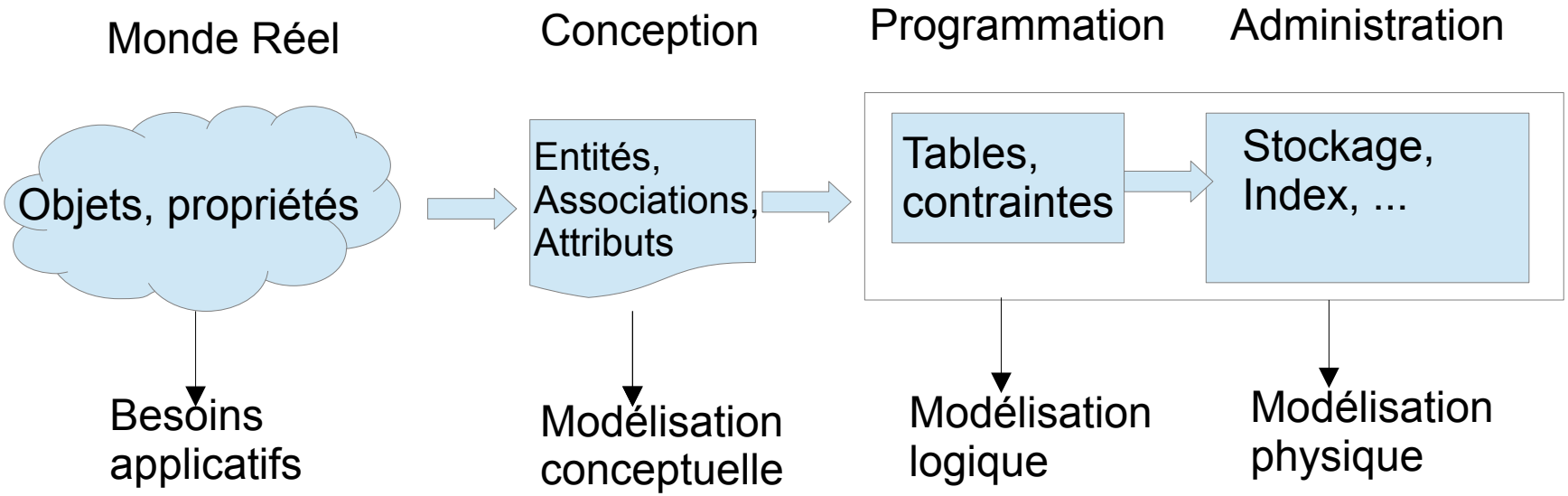
2) Modélisation des données : *formaliser les besoins, conception de données*



3) Implantation des données et des procédures d'interrogation et de mise à jour : *programmation Bases de Données*



Construction d'une base de données



Quels outils ?

- Analyse des besoins :
  - Discussion ‘informelle’ qui découle sur une documentation technique
- Modélisation des données :
  - Traduction des besoins en entités et lien entre elles
  - **Modèle Entite-Association**
- Manipulation des données :
  - Langages spécialisés et facile à utiliser par l’utilisateur

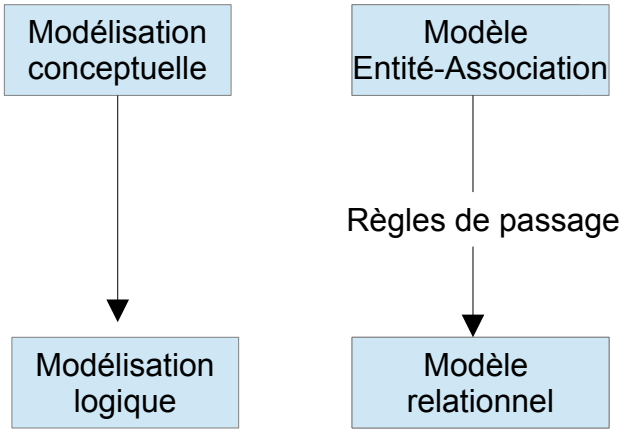
Modèles de Données

Modèle entité-association

- Description haut niveau des données
- Entité et les liens entre elles (associations)
- Interface concepteur - client

Modèles relationnel

- Représentation logique des données
- Tables
- Interface concepteur - développeur



Langages BD

Langages *spécialisés* pour *gérer des données* :

- Comment organiser les données ?
- Comment cherche, supprimer, modifier des données ?

Langages *déclaratifs* pour *faciliter l'utilisation* :

- Spécifier les besoins sans dire comment faire
- Le système génère un programme « optimisé » pour réaliser la tâche demandée

Utilisation :

- Langage de **définition des données** (DDL : *Data Definition Language*)
  - Définition du schéma logique (tables) et physique (index, règles d'accès, ...)
  - Définition des contraintes (cohérence logique)
- Langage de **manipulation des données** (DML: *Data Manipulation Language*)
  - Interrogation des données (requêtes d'interrogation)
  - Insertions, modification et suppression données (requêtes de mises_à-jour)

Cours BD à SU

- LU2IN009 (L2) : Introduction aux Bases de données relationnelles
  - Modélisation et représentation des données
  - Langages de gestion et interrogation des données
  - Programmation

Autres cours BD :

- LU3IN009 (L3) : Systèmes de Gestion de Bases de Données
- MLBDA (M1) : Modèles et Langages pour les Bases de Données Avancées
- SAM (M1) : Stockage et Accès aux Mégadonnées
- LSDA (M2) : Large-Scale Data Analytics
- GDC (M2): Graphes de Données et de Connaissances

Plan du cours

- Modélisation de Bases de Données
  - Modèle Entité-Association
  - Modèle Relationnel
- Interrogation des données
  - Calcul relationnel
  - SQL
- Saisie, modification et cohérence des données
  - Langage de Définition de Données (DDL)
  - Langage de Manipulation de Données (DML)
- Manipulation complexe des données
  - Programmation BD

Conception d’un schéma Entité-Association

Démarche

Analyse des besoins

- Discussion **informelle** avec les futurs utilisateurs
- Identifier les objets du monde réel et des liens entre eux
- Identifier les opérations sur ces objets et les éventuelles évolutions

→ Document technique décrivant les données de l'application

Établissement du schéma conceptuel

- Langage de **haut niveau** (ex. le modèle entité-association)
- Décider des données devant être stockées, de leur propriétés et des relations entre elles
- Définir les contraintes à respecter

→ Schéma dans un langage de haut niveau (Entité-Association)

Etude de cas : BD d'une Université

Les besoins :

- Gérer les inscriptions des étudiants à des modules
- Gérer l'affectation des tuteurs à des étudiants
- Gérer le planning des salles

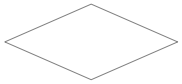
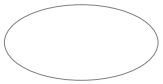
Les objets à modéliser :

- Les étudiants
- Les modules
- Les tuteurs
- Les salles

Les liens entre les objets (exemples) :

- Les étudiants s'inscrivent à un ou plusieurs modules pour une année universitaire
- Le cours d'un module a lieu dans une salle donnée ; il débute à une heure connue et se déroule pendant une durée connue.

Le modèle Entité-Association

- Proposé par Peter Chen en 1976
- **Principe** : Transcrire les besoins en terme de *classes* d'*entités* et de *classes* d'*associations*
- *Composants du modèle :*
  - *Entité* = objet du monde réel. Notation :
  - *Association* = relations entre les objets. Notation : 
  - *Attribut* = caractéristique des objets et relations. Notation : 

Entités

- **Entité = objet** du monde réel **identifiable et pertinent** pour l'application
- Exemple d'entités concrètes :
  - Le bâtiment 24-34
  - La salle 24-34/208
  - La personne Anne Dupont
- Exemple d'entités abstraites :
  - Le module "LU2IN009"
  - Le virement bancaire n° XXX

Attributs d’entités

- **Attribut = propriété/caractéristique** d’une entité
- Chaque attribut
  - possède un nom
  - un domaine de valeurs atomiques est un ensemble de *valeurs simples* : entiers, chaînes de caractère, date, Booléen
- Exemple d'attributs :
  - L’attribut *nom* est une chaîne de caractères
  - L’attribut *matricule* est un nombre entier
  - L'attribut *prénom* est une chaîne de caractères
  - L’attribut *datenaiss* est une valeur de type Date

Classe d'entités

- **Classe d’entité = ensemble nommé d'entités** ayant les mêmes caractéristiques (attributs)
- Exemple de classes d'entités :
  - La classe *Étudiant* décrit l'ensemble des étudiants de l'université (tout étudiant a un nom, prénom, adresse, matricule)
  - La classe *Module* décrit toutes les modules de l'université (tout module a un code, un intitulé et un niveau)
  - La classe *Médecin* décrit tous les médecins de l'application (tout médecin a un matricule et un nom)
  - Etc.

Identifiants d’entités

- **Identifiant** : un sous ensemble d'attributs d'une classe d'entités qui permet de distinguer les entités de la même classe
  - deux entités de la même classe ne peuvent pas avoir la même valeur pour tous les attributs identifiants
  - toute classe d'entités doit avoir un identifiant (si plusieurs identifiants potentiels → en choisir un seul)
- Un identifiant peut être :
  - **naturel** : construit à partir des propriétés / caractéristiques naturelles des entités (nom, prénom, ...)
  - **artificiel** : rajouté aux attributs de la classe lorsque les attributs naturels de la classe ne permettent pas de définir un identifiant

- Notation : <u>att1</u> <u>att2</u> <u>att3</u>

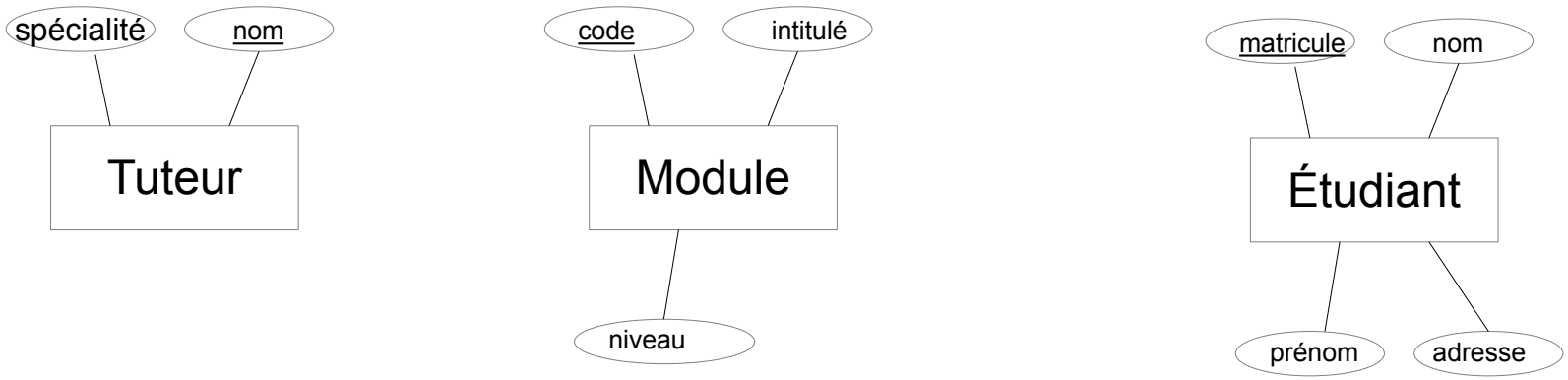
Exemples d'identifiants

- Le *matricule* permet de distinguer l'ensemble d'étudiants de l'université : identifiant de la classe *Étudiant*
- Pour une *salle d'enseignement*, son numéro (e.g 207) ne permet pas de la distinguer des autres salles de l'université (il peut y avoir une autre salle 207) ; on a besoin aussi de connaître son emplacement (e.g 24-34), donc le couple (numéro, emplacement) (e.g (207, 24-34)) sera l'identifiant de la classe *Salle*
- Si chaque étudiant est identifiable par son nom et son adresse (deux étudiants avec le même nom n'habitent pas la même adresse) ou par son matricule, choisir un des deux (e.g matricule) comme identifiant de la classe *Étudiant*
- Si plusieurs enseignants peuvent avoir le même nom et le même prénom (le couple nom, prénom ne peut pas jouer le rôle d'identifiant), alors ajouter un attribut artificiel id à la classe *Enseignant* qui sera désigné comme identifiant.

Étude de cas : BD d'une université

Classes d'entités :

- **Tuteur :** les tuteurs ont un nom (identifiant) et une spécialité
- **Module :** les modules d'enseignements ont un code (identifiant) et un intitulé
- **Étudiant :** les étudiants ont un matricule (identifiant), un nom, un prénom et une adresse

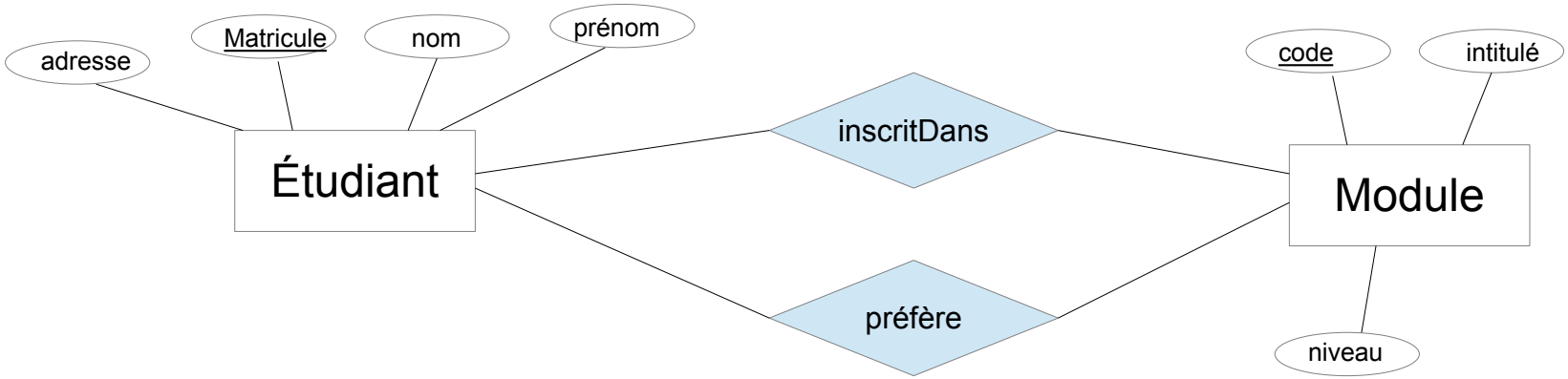


Attributs soulignes (nom, code, matricule) : identifiants

Association et Classe d'Association

- **Association** = relation entre **deux ou plusieurs entités**
- **Classe d'association = ensemble nommé de relations** « du même type » entre des entités
- Exemple d'associations :
  - L'enseignante « Anne DUPONT » enseigne le cours « LU2IN009 »
  - L'étudiant « Axel DUPOND » est ami de l'étudiante « Claire MEUNIER »
  - Le « TD2 » du cours « LU2IN009 » a lieu dans la salle « 25-26/106 »
- Exemple d'associations :
  - La classe d'associations « enseigne » lie des entités de la classe d'entité « Enseignant » à des entités de la classe « Cours »
  - La classe d'association « ami » relie des entités de la classe « Etudiant »
  - La classe d'association « réservation » relie des entités de la classe « TD » aux entités de la classe « Salle »

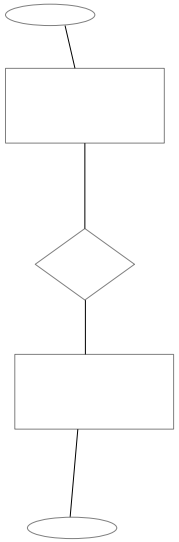
Classe d'associations



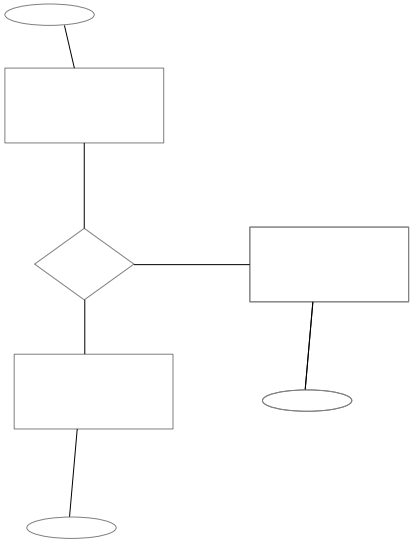
Associations binaires et n-aires

Associations binaires : relient deux entités

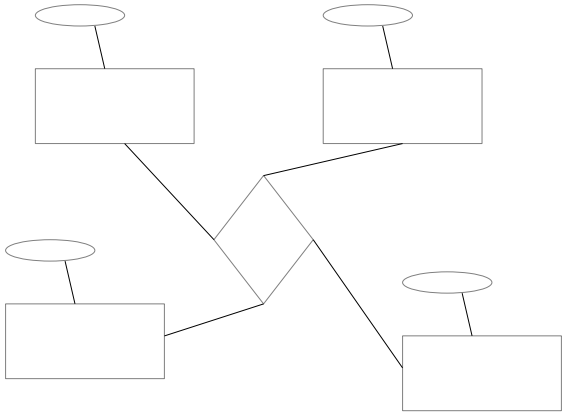
Associations N-aires : relient trois ou plus entités



Association binaire



Association ternaire

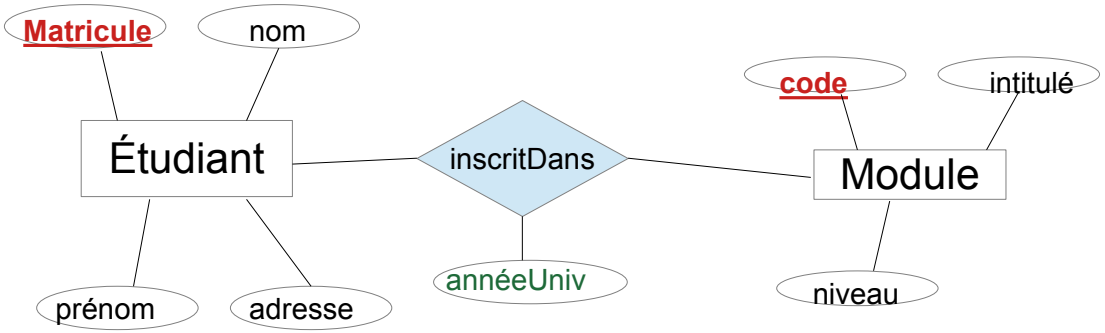


Association quaternaire

Attributs et identifiant d'une association

- Une association peut avoir des **attributs : propriétés qui décrivent la relation** et non pas les entités reliées
- L'**identifiant d'une association** est défini par l'*ensemble des identifiants des entités* qu'elle relie → on ne peut pas associer les mêmes entités plusieurs fois avec des associations de la même classe
- Exemple:
  - l'étudiant 1753 est inscrit au module LI341 *en 2025 (attribut d'association)*
  - l'étudiant 1753 ne peut pas s'inscrire plusieurs fois au module LI341

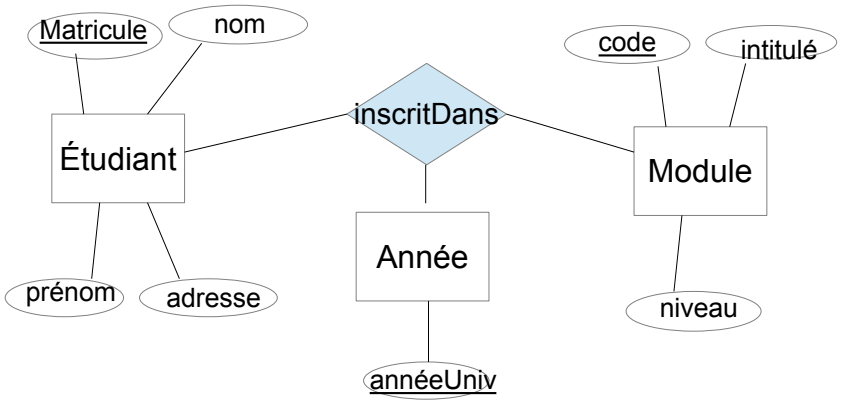
- Identifiantd de *inscritDans* : *{Matricule, code}*
- Attributs de *inscritDans* : annéeUniv



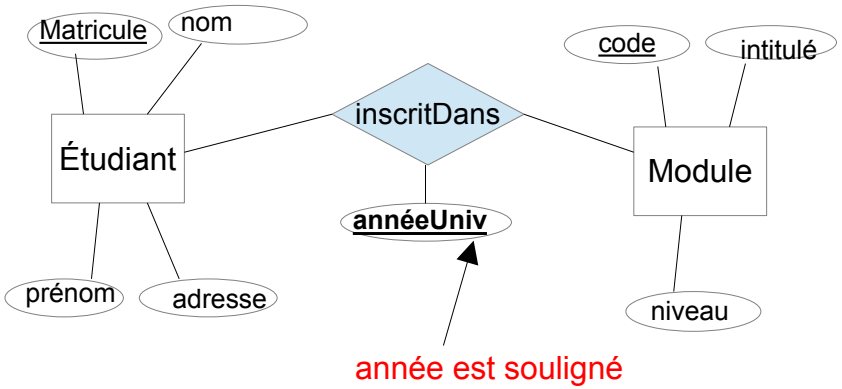
Identifiant d'une classe d'associations

- On souhaite qu'un étudiant puisse s'inscrire à un module *plusieurs fois, à des années différentes*
- Problème : l'identifiant d'une association est composé des identifiants des entités reliées
  - Solution1 : créer une entité « Année » et définir une association ternaire
  - Solution 2 : intégrer l'année dans l'identifiant de l'association

Solution 1



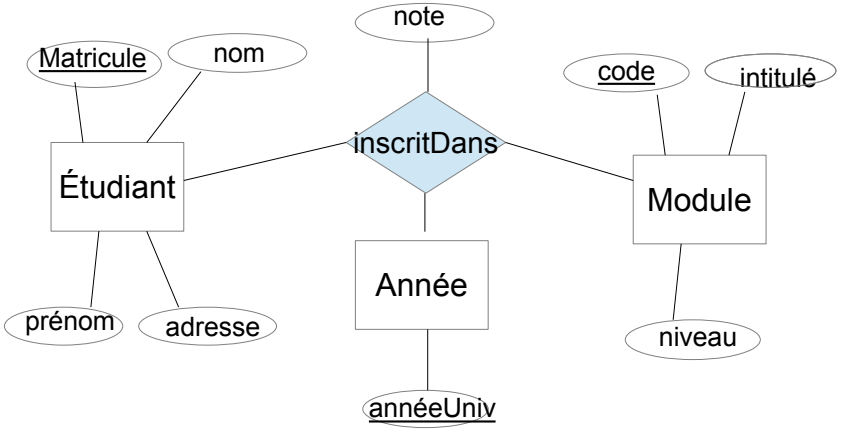
Solution 2



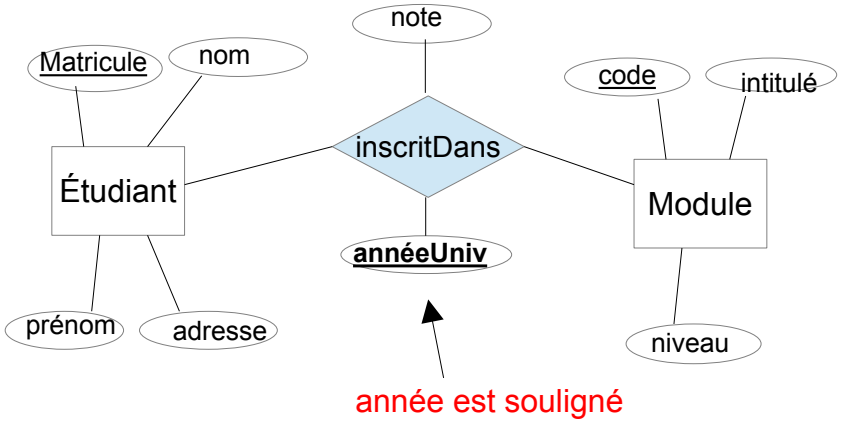
Identifiant d'une classe d'associations

Un étudiant s'inscrit à un ou plusieurs modules pendant <u>une ou plusieurs années</u> et obtient à chaque inscription une note.

Solution 1



Solution 2



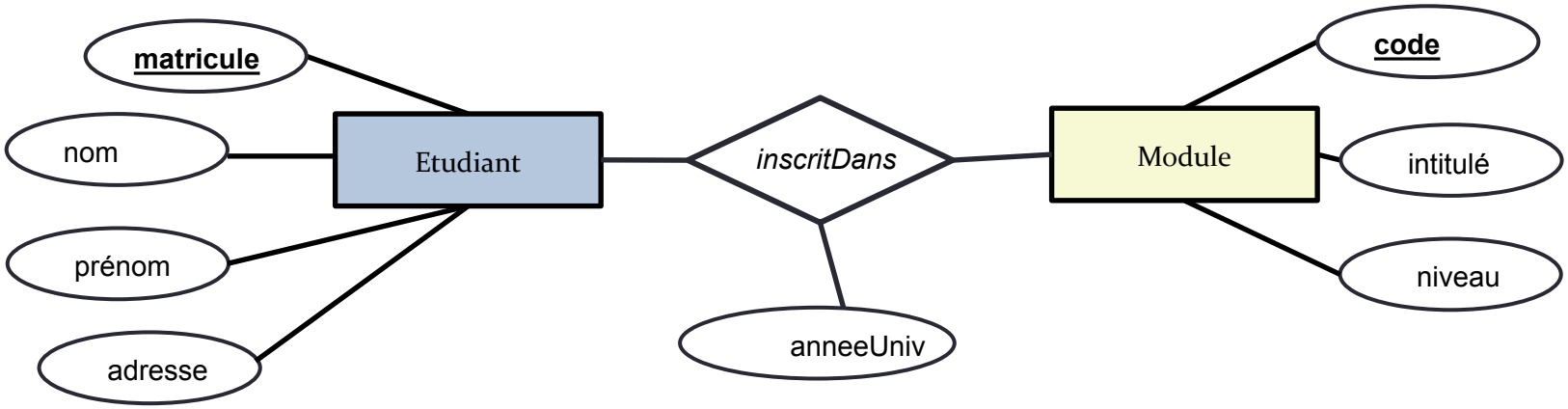
Attention :

- il faut bien justifier l'utilisation des attributs identifiants dans les associations
- Utilisations fréquentes : associations *temporelles* (avec des attributs de type « date »)

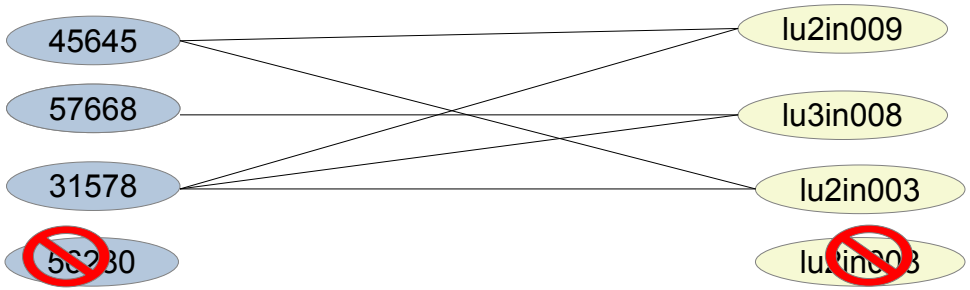
Simplifications de la terminologie

- Observation : Pendant la conception d'un schéma E/A, on décrit principalement les classes d'entités et d'associations (et non pas des entités et associations particulières)
- Simplification de terminologie : souvent (par abus de langage) on omet le terme « classe » :
  - Le terme « entité » sous-entend « classe d'entités »
  - Le terme « association » sous entend « classe d'associations »

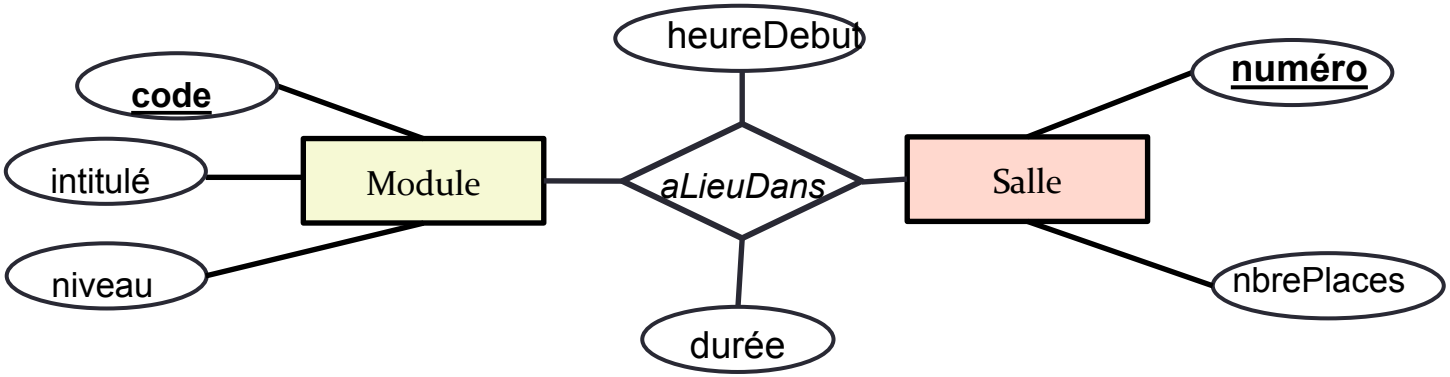
Etude de cas : BD d'une université



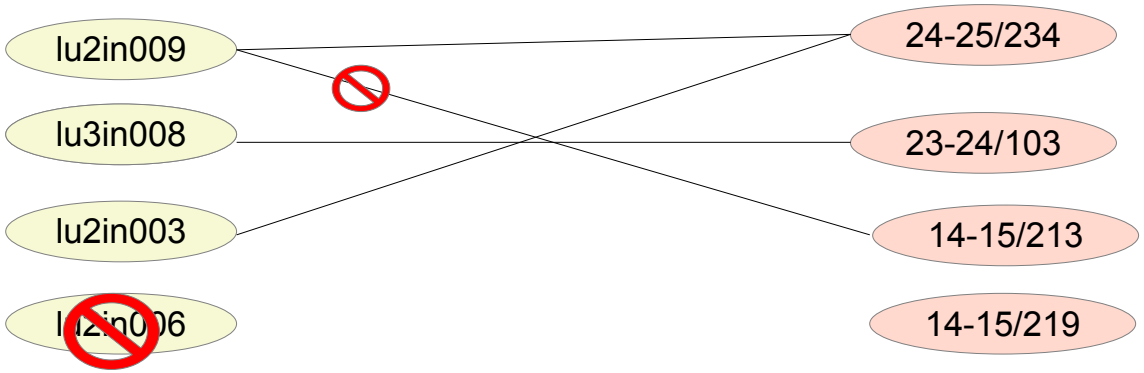
- Un module doit avoir au moins un étudiant inscrit
- Un étudiant doit être inscrit dans au moins un module



Etude de cas : BD d'une université



- Un module a lieu dans une seule salle
- Une salle peut accueillir aucun ou plusieurs modules

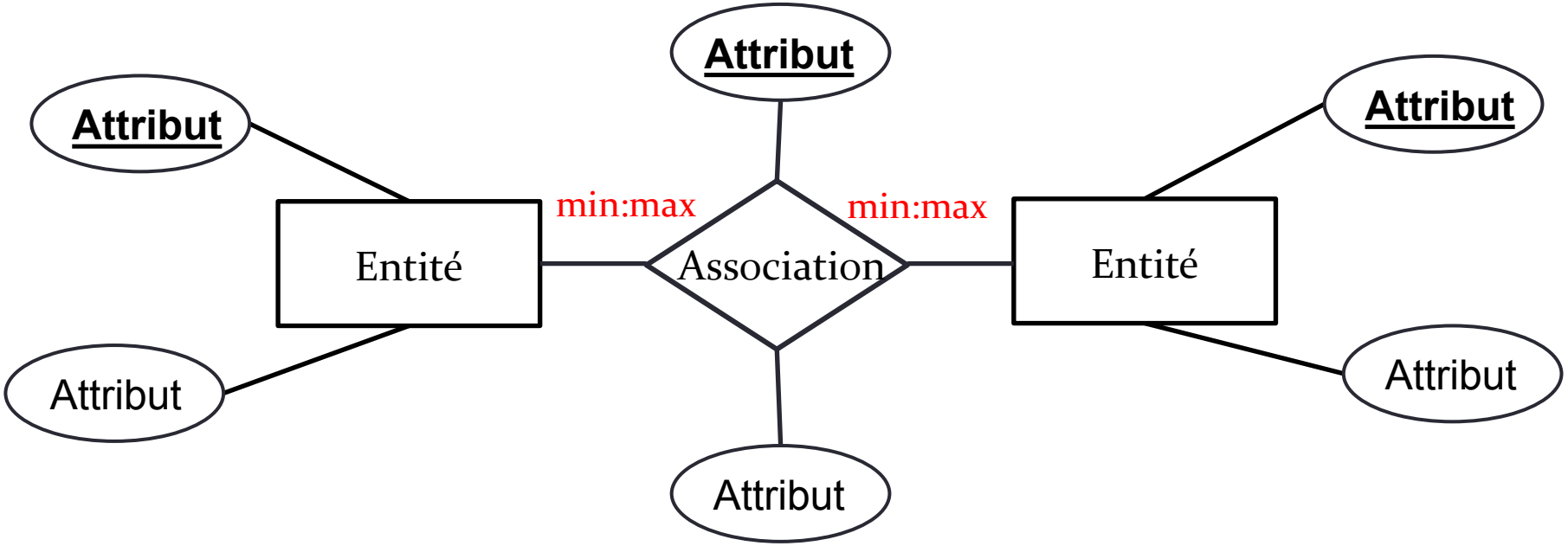


Cardinalités d'une classe d'association

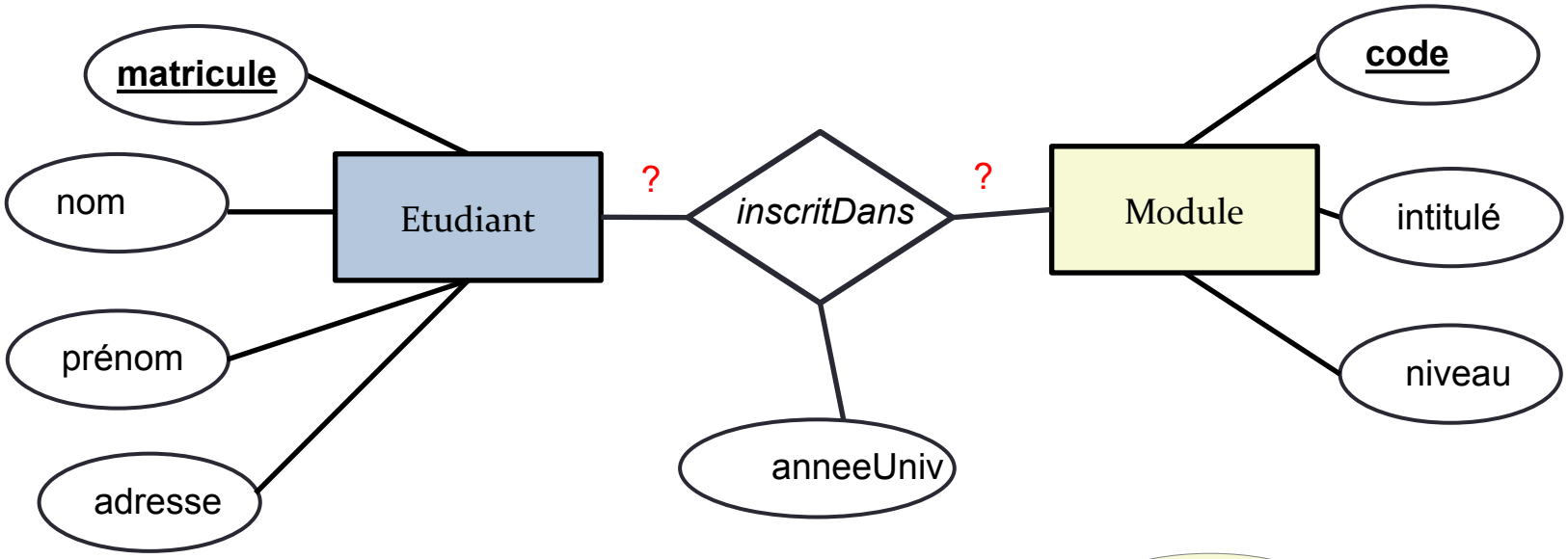
- **Cardinalités d'association:** Pour une classe d'entité E et une classe d'associations A, le nombre minimum (min) et le nombre maximum (max) d'associations de la classe A qui peuvent exister pour une entité (instance) de la classe E.
- Valeurs communes pour min et max: **0**, **1** ou **N** (n'importe quelle valeur)
- Par définition : min <= max et 0<max (sinon aucune association n'est possible)

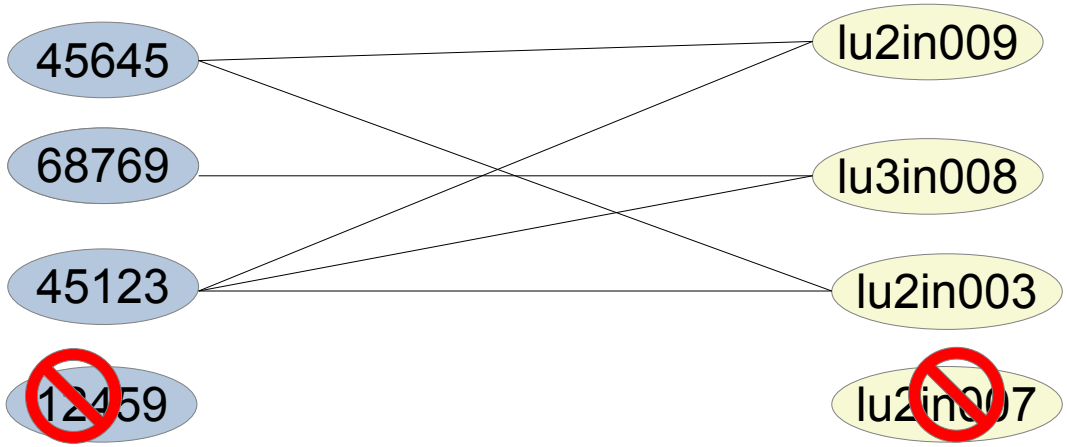
Description	Entités	Association	Cardinalité min:max
Un module a au moins un étudiant inscrit	Module	inscrit	1:N
Un étudiant doit être inscrit dans au moins un module	Etudiant	inscrit	1:N
Un module a lieu dans une seule salle	Module	aLieuDans	1:1
Une salle peut accueillir aucun ou plusieurs modules	Salle	aLieuDans	0:N

Modèle E-A : Notation graphique

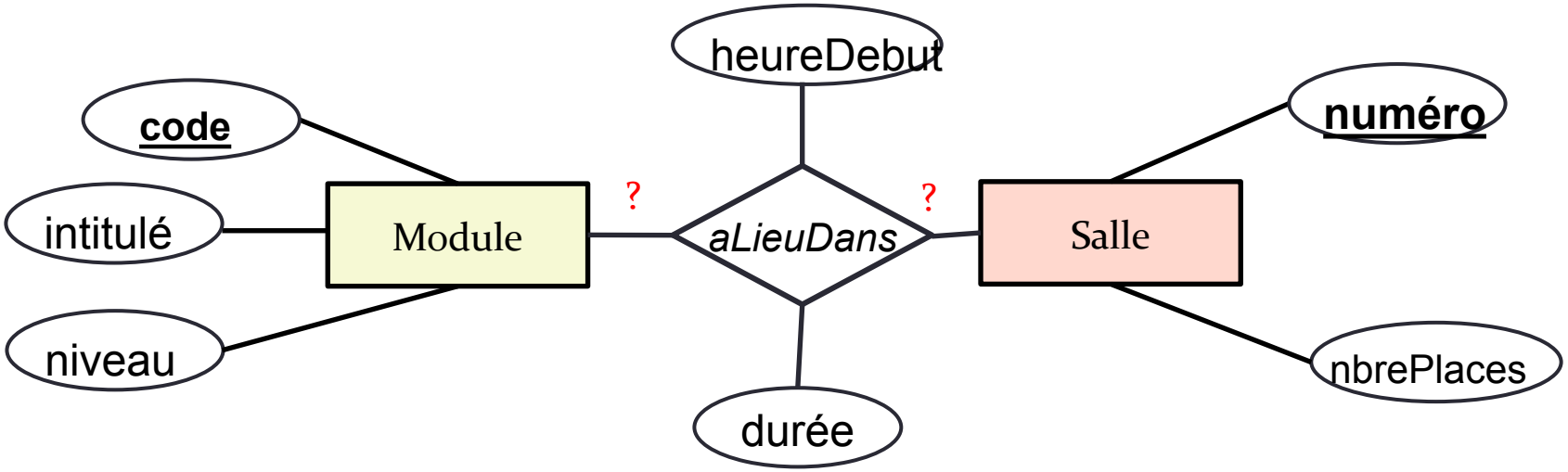


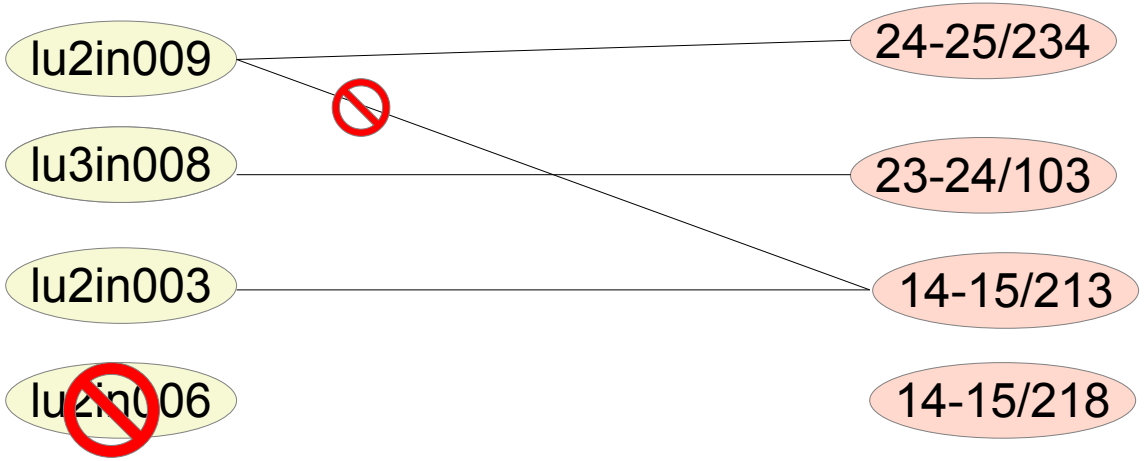
Etude de cas : BD d'une université





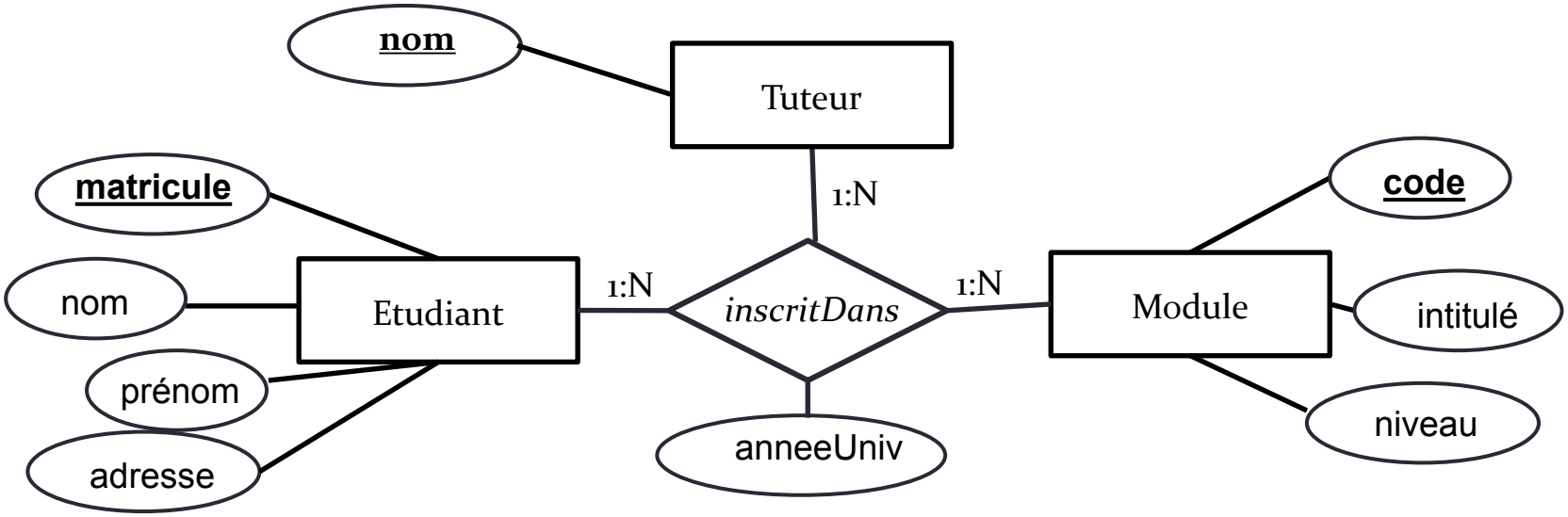
Etude de cas : BD d'une université



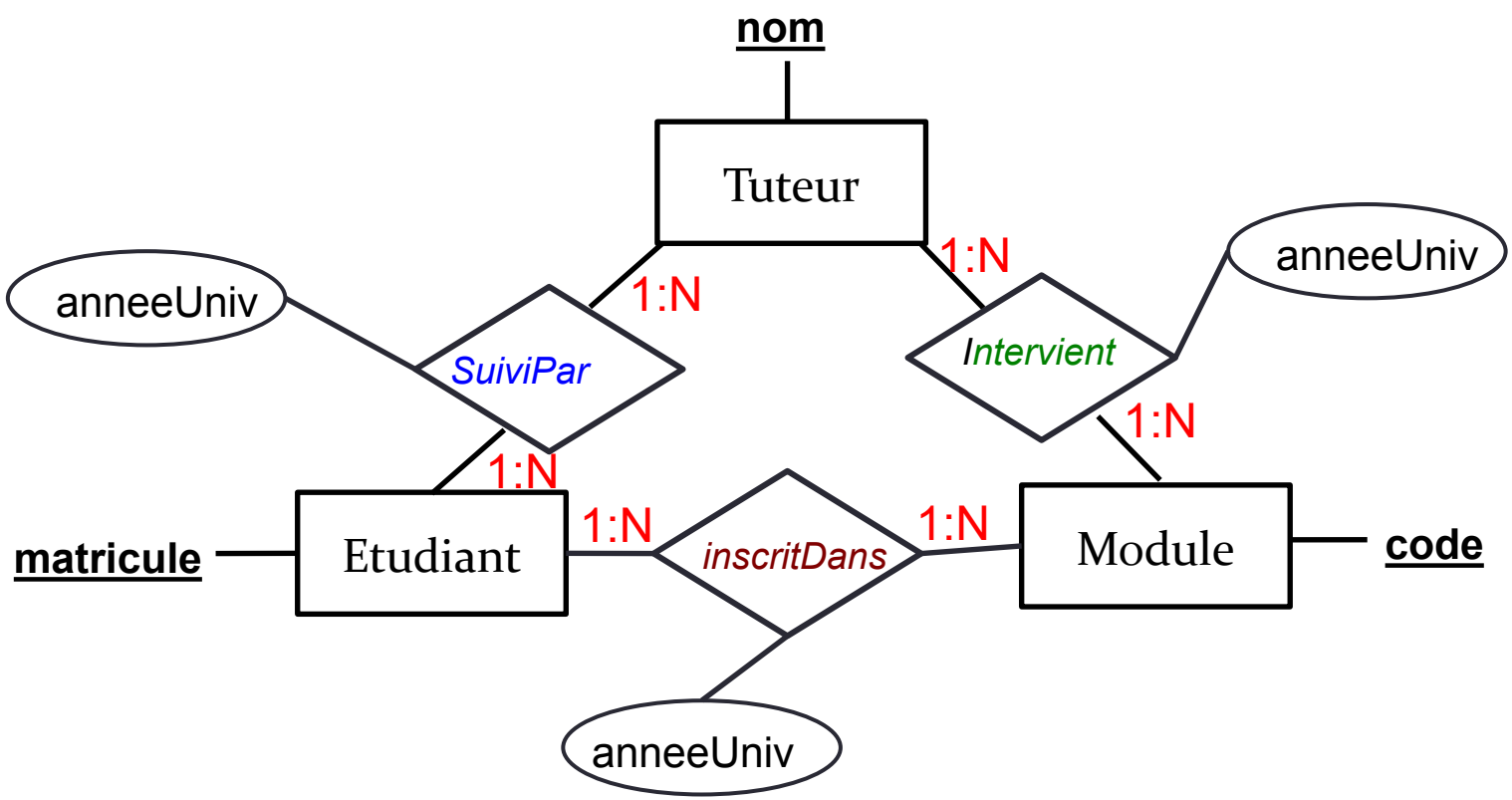


Associations ternaires

- Un étudiant doit s'inscrire dans au moins un module avec un tuteur
- Un tuteur doit tutorer au moins un étudiant dans un module.
- Un module doit avoir au moins un étudiant inscrit avec un tuteur.



Trois associations binaires

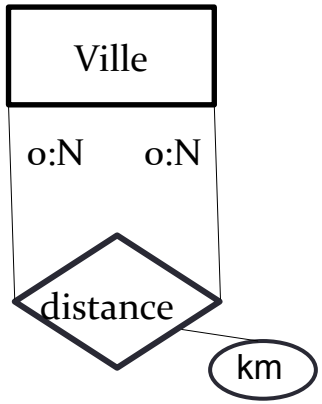


- Un étudiant peut être suivi par un tuteur qui n'intervient pas dans le module où il est inscrit !
- On peut enlever l'inscription d'un étudiant à un module en gardant son tuteur !

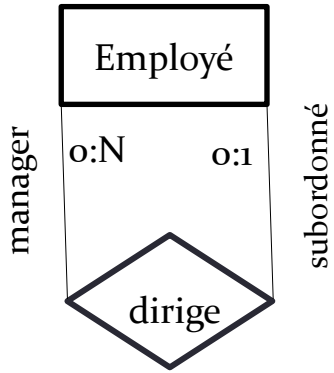
Associations réflexives

- Association **réflexive** : association qui relie des entités de la même classe d'entités
- Association **réflexive asymétrique** :
  - les entités de la même classe jouent des *rôles différents* dans l'association
  - les différents rôles sont identifiés par une *étiquette*
  - les cardinalités peuvent être différents des deux côtés
- Association **réflexive symétrique** :
  - les entités de la même classe jouent le même rôle
  - les cardinalités sont les mêmes des deux côtés

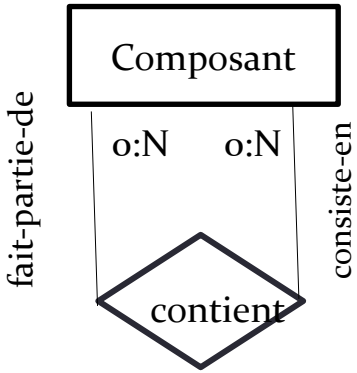
Symétrique



Asymétrique



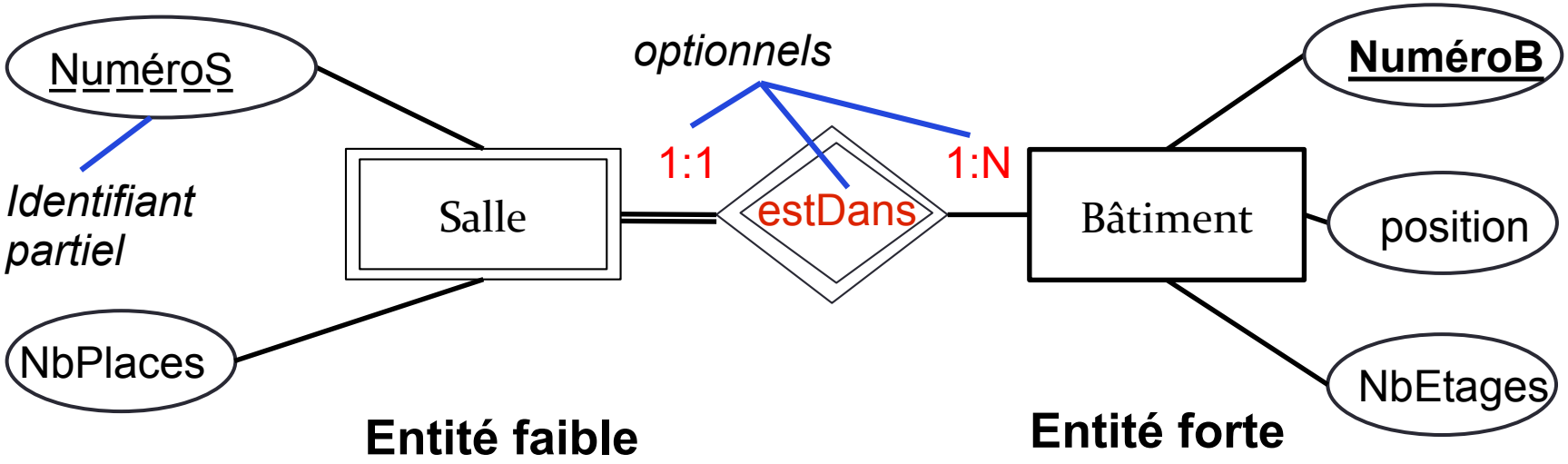
Asymétrique



Entités faibles et entités fortes

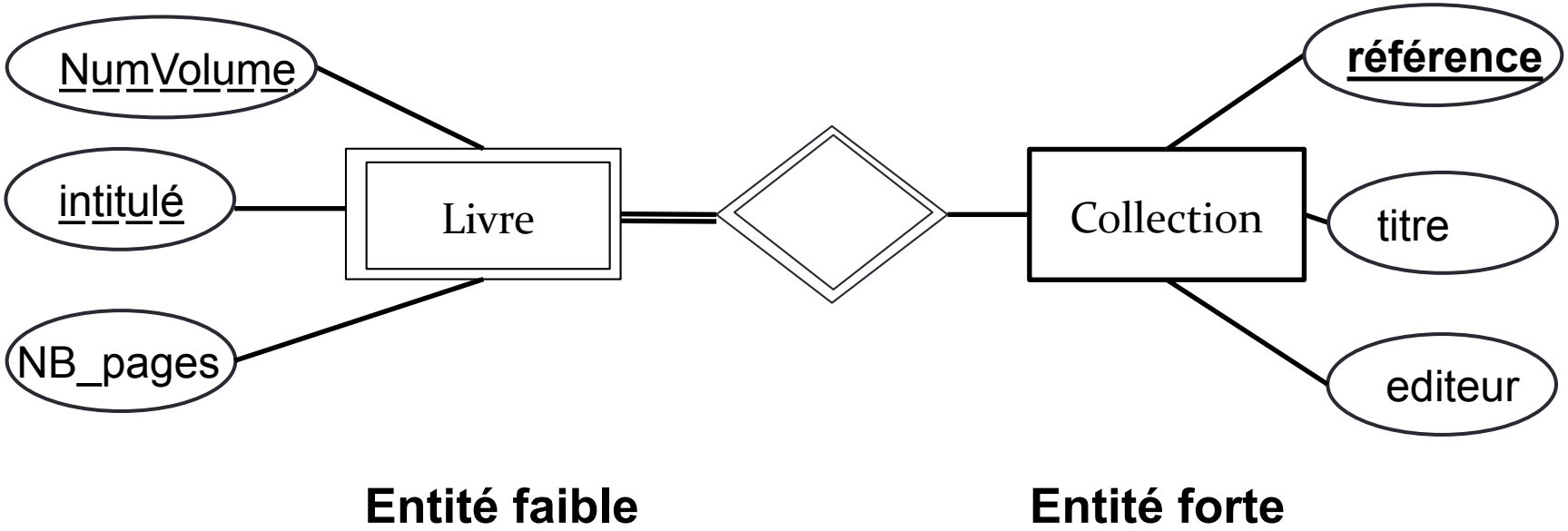
- *Entités faibles* : entités dont l'identifiant dépend de l'identifiant d'autres entités (entités fortes).
- Exemples :
  - La section 3 (entité faible) d'un livre (entité forte)
  - La salle 103 (entité faible) du bâtiment 25-26 (entité forte)
  - Le bâtiment 34-35 (entité faible) du campus Jussieu (entité forte)

Associations
“Entités faibles / fortes”



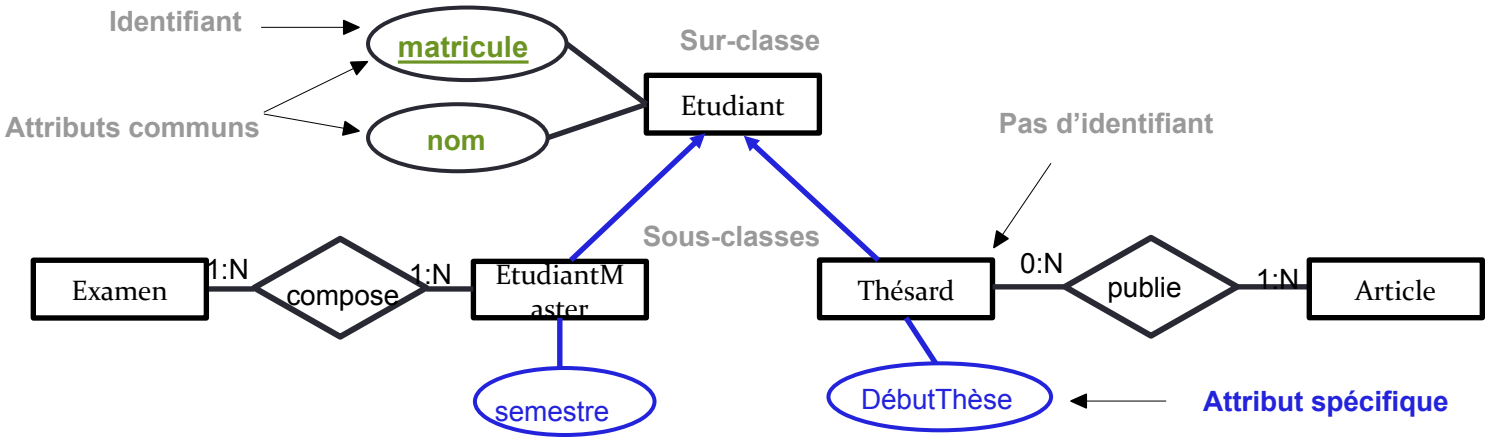
- Les entités faibles possèdent des ***identifiants partiels*** (e.g *NumeroS)* qui sont uniques « dans le contexte » de leur entité forte
- L’**identifiant global** est composé de l’identifiant partiel et de l’identifiant de l’entité forte : (NumeroB, NumeroS)
- Représentation de l’identifiant partiel : attributs soulignés en pointillés

Entités faibles (autre exemple)



Spécialisation et Généralisation

- **Observation**: les entités à modéliser *partagent certaines propriétés* et possèdent d'autres priorités qui leur sont propres
- **Principe de généralisation** : créer une sur-classe d'entité avec les propriétés en commun dont vont *hériter* les sous-classes plus spécifiques
- **Principe de spécialisation** : créer des sous-classes d'entité avec les propriétés spécifiques par rapport à la sur-classe
- **Attention :** la sur-classe *définit l'identifiant de ses sous-classes* (qui n'ont pas d'identifiant !)
- **Exemple** : Les *étudiants en master* passent des examens et les *étudiants en thèse* publient des articles scientifiques



Choix de conception

Analyse des besoins produit une spécification peu précise

- plusieurs choix de conception possibles

Questions fréquentes :

- Un objet du monde réel peut-il être modélise par une entité ou par un attribut ?
- Un objet du monde réel peut-il être modélise par une entité ou par une association ?
- Un attribut décrit-il une association ou une entité?

Choix de conception

Entité ou attribut ?

*Exemple : L'*adresse d'un étudiant

1) est un **attribut** *adresse* de type chaîne de caractère de l'entité étudiant **<u>ou</u>**
2) une **entité** *Adresse*, ayant comme attributs numéro, voie, code postal et relié à l'étudiant par une association ?

Réponse :

- La décision dépend de l'application et son évolution
- Par exemple, si on veut modéliser que des étudiants peuvent partager la même adresse, on peut introduire une entité

Entité ou association ?

Règle générale : toute action impliquant deux ou plusieurs entités donne lieu à une association.

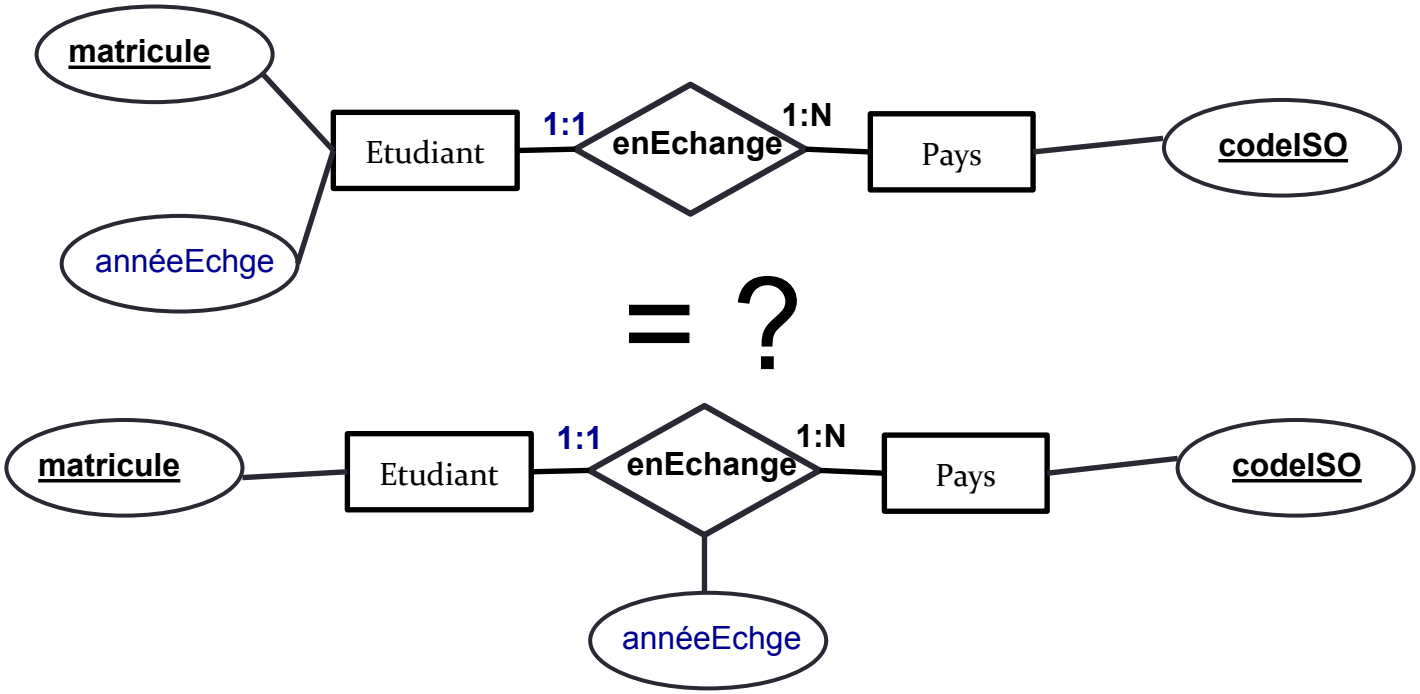
Exemple: (le cours d'un) module a lieu dans une salle → association *ALieuDans*

Choix de conception

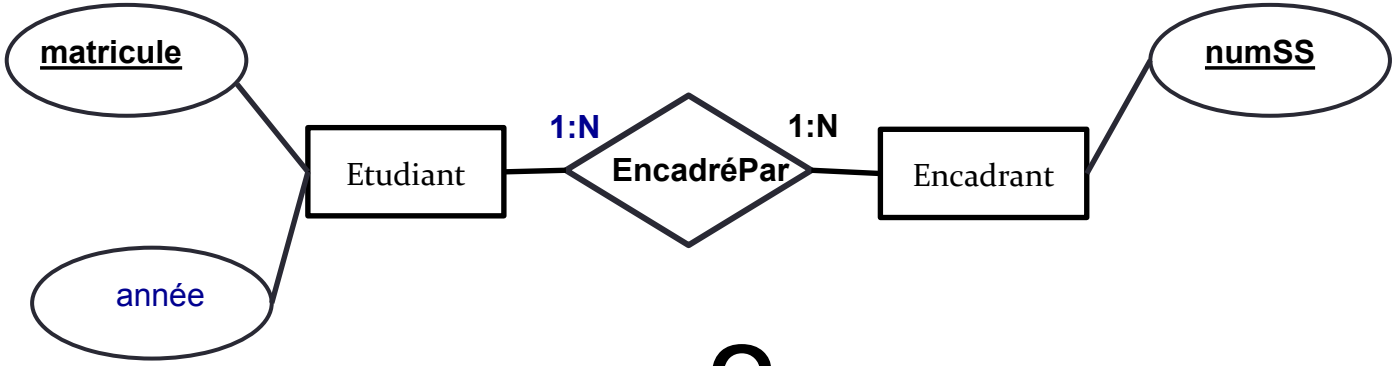
Attribut d'une association ou d'une entité?

Le choix dépend des cardinalités

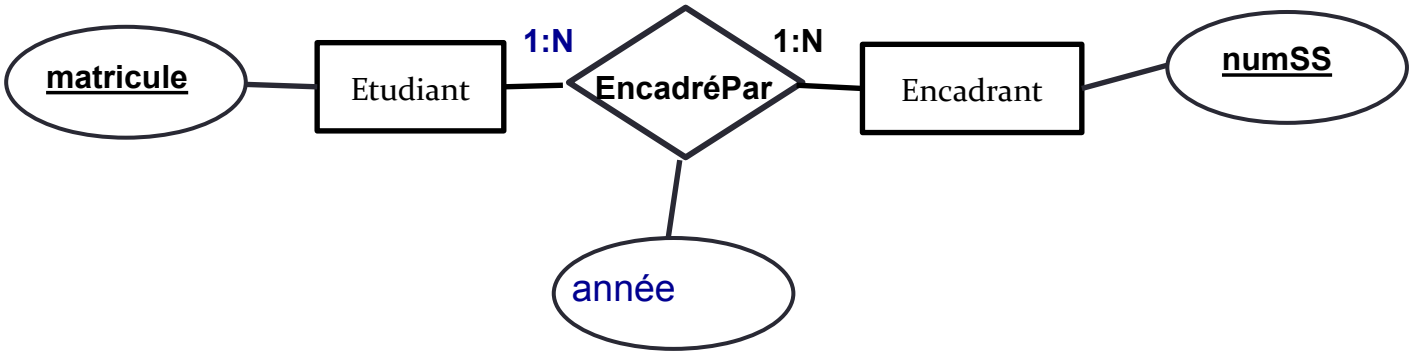
- Cardinalité **1 à plusieurs** : les deux alternatives sont équivalentes puisque l'une des deux entités participe <u>une seule fois</u> dans l'association
- Cardinalité **plusieurs-à-plusieurs** : la sémantique diffère selon le cas où l'attribut est au niveau de l'entité ou de l'association



Attribut d'association ou d'entité?



= ?



Conclusion

- L'intérêt des bases de données
  - Méthodologie pour la conception et la structuration de données
  - Différents niveaux d'abstraction qui permettent l'interopérabilité entre les systèmes
- Plusieurs étapes pour créer une base de données
  - Analyse de besoins
  - Modélisation des données
  - Création des données
- Modélisation des données
  - Transcription de la réalité vers le modèle Entité-Association
  - Plusieurs alternatives
- Suite: passage du modèle Entité-Association vers le modèle relationnel

Bibliographie conseillée

Notes de cours

- S. Gancarski. Introduction aux bases de données. UPMC, Paris 6, janvier 2003 – lien sur le site de l'UE

Livres en anglais

- R. Ramakrishnan and J. Gehrke. Database Management Systems 3e édition, McGraw Hill, 2002 -http://pages.cs.wisc.edu/~dbbook/ (*Disponible bib. MIR et MIE)*
- A. Silberschatz, HF. Korth and S. Sudarshan. Database System concepts 6e édition, McGraw Hill, 2011-http://db-book.com/

Livres en français :

- S. Abiteboul, R. Hull, V. Vianu, Les fondements des bases de données, Vuibert (*Disponible bib. MIR et MIE)*
- G. Gardarin. Bases de données - objet et relationnel. Eyrolles. (*Disponible bib.* L1-L2 scientifique *et MIE)*